

Codeforces Mashup Generator

A Brief

Explainer for Salman Adnan's portfolio

1 The problem

A study group ran practice programming contests by assembling them from existing Codeforces problems. Codeforces calls such an assembled contest a *mashup*. The contests kept being spoiled the same way: someone would open a problem they had already solved months earlier, and for them the contest was over before it started.

Picking problems by hand does not scale past a few people. Codeforces stores roughly 11,000 rated problems and a permanent public record of every submission every user has ever made, so the fix is mechanical: ask the site what everyone has already solved, and never pick those.

What the tool is

One Python script, 228 lines, three dependencies. It asks the public Codeforces API for the whole problem catalogue and for each group member's complete submission history, discards every problem any member has already solved, optionally keeps only problems a strong competitor has solved, and writes what survives to an Excel sheet grouped by difficulty. A human then builds the actual mashup from that sheet. The tool selects; it does not create the contest.

2 How it works

Two API endpoints supply everything. They are the reason the design looks the way it does, because they return the same logical entity in two incompatible shapes.

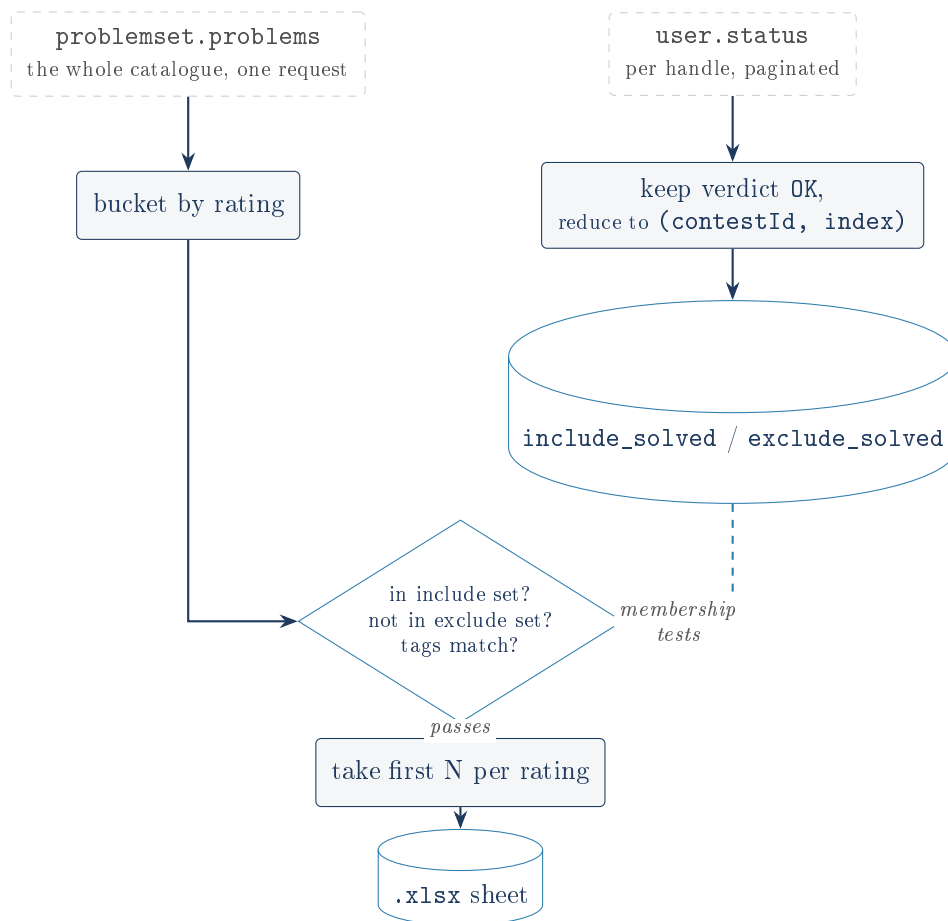


Figure 1: The pipeline, end to end. It is a straight line: no persistence, no caching, no state between runs.

The design decision worth defending

`problemset.problems` returns problem objects with `contestId` and `index` at the top level. `user.status` returns *submission* objects that wrap the problem one level down, under a `problem` key. The two cannot be compared directly.

The fix is to choose a **canonical identity**, the tuple `(contestId, index)`, and reduce both sources to it before doing anything else. Once both sides speak the same identity, filtering stops being a search problem and becomes set arithmetic: one constant time lookup per problem, against sets built once. This is why there is no nested loop anywhere in the filtering code, and it is the idea that generalises past this project.

A second decision is worth naming. Include and exclude need an identical computation over different handle lists. That started as one loop copy-pasted twice, meaning a fix to the verdict rule had to be applied in two places or it was wrong in one. It is now a single `collect_solved(handles, delay)` called once per list, so the “solved means verdict OK” rule exists in exactly one place. The duplication was not just untidy, it was a correctness hazard.

3 Configuration

Everything is a command line flag; there is no config file and no environment variable. The main controls are the two handle lists (`--include-users`, `--exclude-users`, each disableable), the rating bands (`--ratings`, default 800 to 2000 in steps of 100), how many options per band (`--choices-per-rating`, default 10), and tag filtering.

Tags support four matching modes: **some** (at least one requested tag), **all** (every requested tag, extras allowed), **strict** (exact tag set, no extras), and **off**. `--exclude-tags` is deliberately independent of the mode and always applies, which is correct: “how should I match these tags” is a meaningful question for tags you want and a meaningless one for tags you want to avoid.

4 Verified, not assumed

The README’s claims were checked against the live API on a clean Python 3.12.3 environment rather than taken on trust. The documented example command returned **9 rows, 3 at each requested rating, all tagged greedy, in 12.1 seconds** against a claimed 14. The handle lists are 46 trusted and 34 excluded as stated, with no duplicates and no overlap between the two. Bad handles do print an error and skip, with the sheet still generating, as documented.

The README is accurate

Every checkable claim in it held. Nothing was found to be overstated, and its “What I’d do differently” section independently identifies most of the real weaknesses below. That is unusual and it is the best signal in the repository.

5 Honest limits

A failed fetch silently voids the entire guarantee

This is the one that matters. Every error path in the submission fetcher prints a message and returns whatever it has, which on an early failure is nothing. The caller cannot distinguish “this member has solved nothing” from “we failed to learn what this member solved”. Both are the empty set.

So a network blip or a renamed handle for one group member means that member’s solved problems are treated as unsolved, they pass the exclude filter, and they get handed a problem they have already done: exactly what the tool exists to prevent. The run prints one line and exits 0. I reproduced the mechanism live with an invented handle.

The severity is asymmetric, and the code does not reflect it. A failed *include* fetch costs some candidates, a quality loss. A failed *exclude* fetch costs the guarantee, a correctness loss. They are handled identically. The exclude case should abort the run.

Selection is truncation, and it is biased

Choosing N problems per rating is `plist[:n]`: the first N in API order, not a sample. Codeforces returns newest-first, so the tool systematically hands back recent problems and never an older classic, and repeat runs with the same filters produce near-identical sheets, which undercuts a tool built for a recurring group. My live run returned contest ids clustered in 2157 to 2237, out of a catalogue reaching back to the low hundreds. The bias is total, not marginal. `random.sample` with an optional seed is roughly a four line fix.

The include filter downloads hundreds of megabytes to answer a yes/no

The include filter’s only purpose is a rough quality signal: has a strong competitor bothered with this problem. To get it, the tool downloads the complete lifetime submission history of 46 people. I measured three of them live: 3.1, 5.2 and 5.6 MB for a *single* page each, which extrapolates to roughly **214 MB as a lower bound** across the default include list, before counting the 34 exclude handles. Nothing is cached, so an identical rerun downloads all of it

again.

Codeforces already returns `solvedCount` on each problem, in data the tool downloads anyway, which approximates the same signal for free. The README names this as the top thing it would change.

No tests, and no exit code discipline

There is no test file and no test dependency, though `passes_tag_filter` and `pick_choices_per_rating` are pure, deterministic, network-free functions that would be straightforward to pin down. Separately, every outcome exits 0 and reports via `print` to `stdout`: success, partial data, and a total failure to fetch the catalogue are indistinguishable to anything downstream. A completely failed run still prints “Done.”. The script is not safely automatable as written.

Dead data at the project root

`all_users.txt` holds two Python lists and looks like configuration. The script does no file I/O at all and never reads it. Comparing it to the script’s own constants dates it: the include lists are byte-identical, but the file’s exclude list has 28 handles against the script’s 34 and is a strict subset of it, missing exactly six. **Confirmed:** it is unread and stale. **Inferred:** it is a pre-refactor leftover rather than a planned input; the code cannot show intent. It should be deleted.

6 Glossary

Codeforces

A large competitive programming site. Hosts the problems and a permanent public record of every submission, exposed through a free, keyless API.

Mashup

A private practice contest assembled from existing Codeforces problems rather than newly written ones. What this tool feeds.

Rating

Codeforces’ difficulty number for a problem, roughly 800 (easiest) to 3500. The axis the tool organises its output around.

Tags

Technique labels on a problem: `greedy`, `dp`, `bitmasks`, `graphs`. Most problems carry several.

Verdict / OK

The judge’s ruling on a submission. `OK` means accepted, that is, genuinely solved. The tool counts only `OK`, so a problem someone attempted and failed stays eligible.

API

A machine-readable door into a website: request a URL, receive structured data instead of a styled page.

JSON

The text format that structured data arrives in, made of nested lists and labelled fields.

Pagination

Returning a huge result in fixed-size slices. Codeforces uses `from` and `count`; the tool requests 10,000 at a time and stops when a page comes back short.

Set

A collection that answers “do you contain this?” in constant time via hashing, rather than by scanning. The reason the filtering is fast.

pandas / openpyxl

Python’s tabular data library, and the engine it delegates to for writing `.xlsx` files. `openpyxl` is required but never imported directly.

argparse

Python’s standard command line parsing library. Generates the `--help` text from the flag declarations.